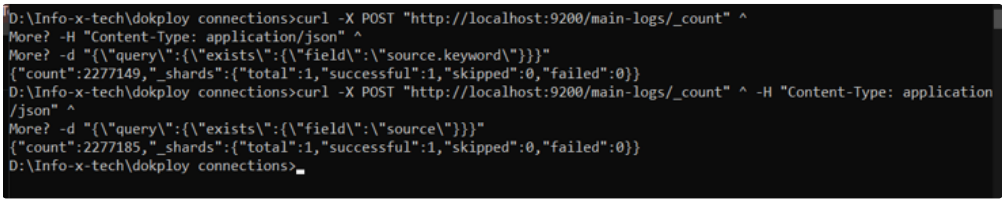
OpenSearch Error Volume Issue, Root Cause and Correct Fix

---

1. What is the issue

We have two environments:

- Local → working correctly



-

- Remote → aggregations returning empty results



-

Both are querying the same field:

```
"field": "source.keyword"

```

But results differ.

What we observed

Local

- `source.keyword` exists in almost all documents
- Aggregations work

Remote

- `source.keyword` exists in 0 documents
- Aggregations fail

Actual problem

Remote index was created without `source.keyword`.

So even though documents contain:

```
"source": "JwtUtil"

```

OpenSearch never created the exact-match version (`source.keyword`) for aggregation.

---

2. How OpenSearch stores data

When you send a document:

```
{
  "source": "JwtUtil"
}

```

OpenSearch stores it in two different ways (depending on mapping):

1. `_source` (raw data)

This is the original JSON stored exactly as sent.

```
"source": "JwtUtil"

```

2. Indexed fields (used for search and aggregation)

These depend on mapping.

---

3. What is mapping

Mapping defines how each field is stored and used.

Example:

```
"source": {
  "type": "text",
  "fields": {
    "keyword": {
      "type": "keyword"
    }
  }
}

```

This means:

- `source` → used for search (text)
- `source.keyword` → used for aggregation (exact value)

---

4. Field types explained

text

- Used for searching
- Data is broken into tokens

Example:

```
"User Service"
```

Stored as:

```
"user", "service"
```

Use case:

- full text search

Not suitable for:

- aggregation
- grouping

---

keyword

- Stores exact value
- Not tokenized

Example:

```
"User Service"
```

Stored as:

```
"User Service"
```

Use case:

- aggregation

- sorting
- filtering

Other common types

Type	Purpose
keyword	exact match, aggregation
text	full text search
integer	numeric values
long	large numbers
double	decimal numbers
boolean	true/false
date	timestamps
object	nested JSON
nested	array of objects (advanced use)

---

5. What is indexing

Indexing means:

> Storing a document in OpenSearch in a way it can be searched and aggregated.

When a document is indexed:

- `_source` is stored
- fields are processed based on mapping
- indexes are created internally

---

6. What is reindexing

Reindexing means:

> Copying data from one index to another with a new mapping.

Process:

1. Read document from old index
2. Apply new mapping
3. Store in new index

---

7. Why reindexing is needed here

Remote index was created without `source.keyword`.

That means:

- Aggregation field does not exist
- Queries fail even though data exists

Reindexing fixes this by:

- Reprocessing all documents
- Applying correct mapping
- Creating `source.keyword` for all documents

---

8. What is the correct solution

Step 1

Create a new index with correct mapping

Step 2

Reindex data from old index to new index

Step 3

Switch application to new index

---

9. Why we do not modify existing index

OpenSearch does not allow changing field types after data is indexed.

You cannot convert:

```
text → keyword

```

on an existing field.

That is why we create a new index.

---

10. Why this is the best solution

- Ensures consistent data
- Fixes all aggregation issues
- Aligns with production standards
- Works for all future queries
- No workaround logic needed in code

---

11. Other possible solutions and their problems

Option 1: Use `source` instead of `source.keyword`

Problem:

- text fields are tokenized
- aggregation becomes incorrect

Example:

```
"User Service"

```

Result:

```
"user": 100
"service": 100

```

This is wrong.

Option 2: Enable fielddata on text field

```
"fielddata": true

```

Problems:

- high memory usage
- slow queries
- not recommended in production

Option 3: Change code to handle both fields

Example:

- try `source.keyword`
- fallback to `source`

Problems:

- inconsistent results
- complex logic
- still incorrect aggregation

---

12. Are there any cons of reindexing

Minor cons

1. Time consuming for large data
2. Temporary increase in storage (old + new index)
3. Requires coordination during deployment

Important note

If done correctly:

- no downtime
- no data loss
- no API impact

---

13. What happens to APIs during reindexing

If handled properly:

- APIs continue working
- only index name changes
- no logic change required

Best practice:

- use index alias
- switch alias after reindex

---

14. What is old vs correct structure

Old (remote)

```
"source": {
  "type": "text"
}

```

Problem:

- no aggregation support

Correct (production standard)

```
"source": {
  "type": "text",
  "fields": {
    "keyword": {
      "type": "keyword"
    }
  }
}

```

---

15. Final summary

- Issue is not in code
- Issue is in index mapping
- Remote index missing `source.keyword`
- Aggregation fails because field does not exist
- Reindexing is required to fix mapping
- This is standard and correct approach

---

16. One line understanding

OpenSearch needs keyword fields for aggregation, and since remote data was indexed without it, we must rebuild the index so those fields are created correctly.

This is everything needed to understand the issue, the system, and the fix end to end.